

# Cahier des spécifications techniques

## Application web de gestion et de suivi d'un élevage cunicole

Version : 2.0 — Spécification consolidée

---

## 1. Présentation générale

### 1.1 Nom provisoire

**Rabbit Farm Management**

Nom commercial à définir ultérieurement.

### 1.2 Objectif

L'application a pour objectif de permettre à un éleveur cunicole de gérer et suivre son élevage de manière centralisée, depuis la reproduction jusqu'à la vente, tout en conservant une **traçabilité historique et généalogique des animaux**.

L'application doit permettre à l'éleveur de savoir à tout moment :

- quels lapins sont présents ;
- où se trouve chaque lapin ;
- d'où vient chaque lapin ;
- quels sont ses parents ;
- dans quelles portées il est né ;
- quelles sont ses performances ;
- quels accouplements ont été réalisés ;
- quelles femelles sont gestantes ;
- quelles mises bas sont prévues ;
- quelles portées doivent être sevrées ;
- quels lapins sont malades ou sous traitement ;
- quels animaux sont destinés à la reproduction ;
- quels animaux sont en engraissement ;
- quels animaux ont été vendus ;
- combien l'élevage dépense ;
- combien il génère ;
- quelle est la situation du stock d'aliments ;

- et quelles actions l'éleveur doit effectuer aujourd'hui.

---

## 2. Vision fonctionnelle

Le principe central de l'application est :

**L'identité du lapin suit l'animal, et non sa cage, sa portée ou son groupe.**

Une cage représente uniquement un **emplacement physique à un instant donné**.

Une portée représente **l'origine familiale d'un groupe de lapereaux**.

Un lot représente **un groupe de gestion**, notamment pour l'engraissement.

Un lapin représente **un individu traçable**.

Cette distinction est fondamentale.

---

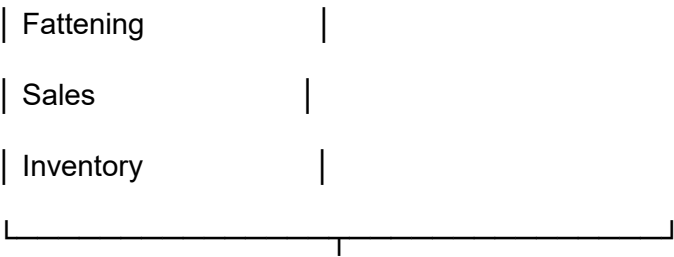
## 3. Architecture générale

L'application sera une **Progressive Web App (PWA)** accessible depuis :

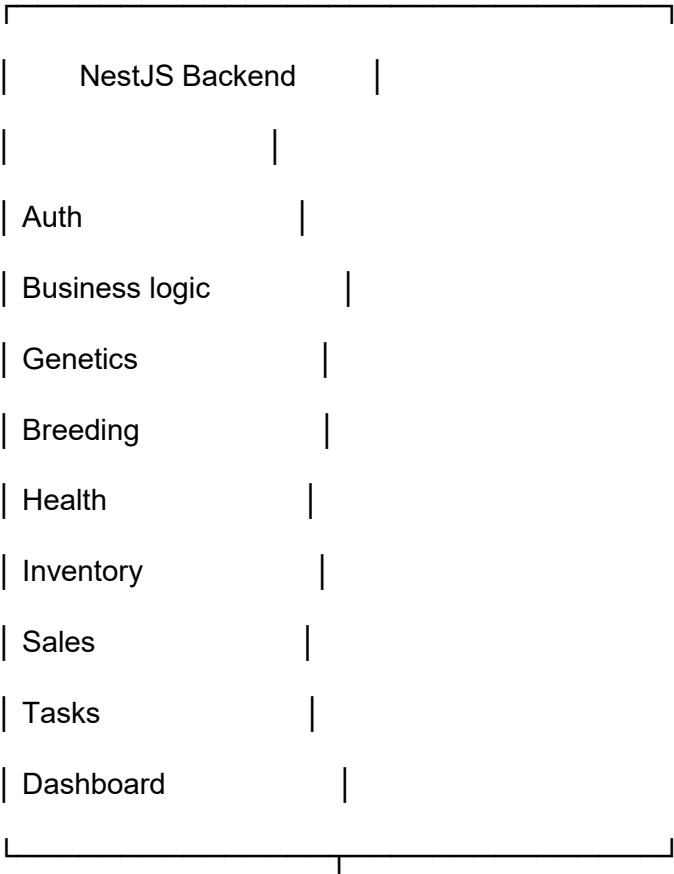
- ordinateur ;
- smartphone ;
- tablette.

Architecture cible :

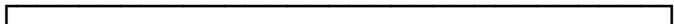
|               |  |
|---------------|--|
| Next.js / PWA |  |
|               |  |
| Dashboard     |  |
| Rabbits       |  |
| Breeding      |  |
| Litters       |  |
| Health        |  |

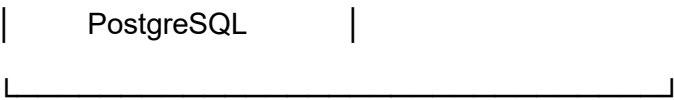


GraphQL



Prisma





Technologies

| Couche             | Technologie |
|--------------------|-------------|
| Frontend           | Next.js     |
| Interface          | React       |
| Type d'application | PWA         |
| Backend            | NestJS      |
| API                | GraphQL     |
| ORM                | Prisma      |
| Base de données    | PostgreSQL  |
| Authentification   | JWT         |
| Identifiants       | UUID        |
| Déploiement        | Cloud       |
| Versionnement      | Git/GitHub  |

---

## 4. Pourquoi GraphQL ?

GraphQL sera utilisé comme API principale.

L'objectif n'est **pas** de créer les relations entre les entités.

Les relations sont gérées par :

PostgreSQL



Prisma

GraphQL permet ensuite au frontend de parcourir ces relations et de demander précisément les données nécessaires.

Par exemple :

```
query {  
  rabbit(id: "...") {  
    code  
    breed  
  
    father {  
      code  
    }  
  
    mother {  
      code  
    }  
  
    breedingAsFemale {  
      breedingDate  
    }  
  }  
}
```

```
male {  
  code  
}
```

```
pregnancy {  
  result  
}
```

```
birth {  
  birthDate  
  liveBorn  
}  
}
```

```
healthRecords {  
  date  
  diagnosis  
  treatment  
}  
}  
}
```

GraphQL sera particulièrement utile pour :

- les fiches détaillées des lapins ;
- les arbres généalogiques ;
- le dashboard ;
- les portées ;
- les statistiques ;
- les relations entre reproduction, gestation, naissance et sevrage.

---

## 5. Utilisateurs et rôles

L'application doit gérer plusieurs utilisateurs.

### 5.1 Administrateur de ferme

Peut :

- gérer la ferme ;
- gérer les utilisateurs ;
- gérer les lapins ;
- gérer la reproduction ;
- gérer les stocks ;
- gérer les finances ;
- consulter les statistiques.

### 5.2 Éleveur / responsable reproduction

Peut :

- gérer les reproducteurs ;
- enregistrer les accouplements ;
- suivre les gestations ;
- enregistrer les mises bas ;
- gérer les portées ;
- gérer les sevrages ;
- consulter la généalogie.

### 5.3 Employé

Peut :

- consulter les tâches ;
- enregistrer les soins ;
- enregistrer certains mouvements ;
- effectuer les pesées ;
- participer au suivi quotidien.

### 5.4 Vétérinaire

Peut :

- consulter les animaux ;
  - enregistrer les observations ;
  - enregistrer les diagnostics ;
  - enregistrer les traitements ;
  - enregistrer les résultats d'analyses.
- 

## 6. Gestion de la ferme

Une ferme possède :

- un nom ;
- une description ;
- une localisation ;
- plusieurs sections ;
- plusieurs cages ;
- plusieurs utilisateurs ;
- plusieurs lapins ;
- plusieurs lots ;
- plusieurs clients ;
- plusieurs ventes ;
- plusieurs dépenses ;
- plusieurs stocks.

La relation :

User ↔ Farm

sera gérée par une table *FarmMembership*.

Cela permet à plusieurs utilisateurs de travailler dans la même ferme.

---

## 7. Gestion des sections et cages

### 7.1 Sections

Types possibles :

REPRODUCTION

MATERNITY

FATTENING

QUARANTINE

OTHER

## 7.2 Cages

Chaque cage possède :

- identifiant ;
- code ;
- section ;
- type ;
- capacité ;
- statut.

Exemple :

Section : Reproduction

C01

C02

C03

...

---

## 8. Principe fondamental : traçabilité des cages

Une cage ne doit **jamais devenir l'identité d'un lapin**.

Exemple :

R034

peut avoir :

01/09 → C03

15/09 → C07

30/09 → C12

15/10 → C05

L'application conserve tous ces mouvements.

Chaque mouvement contient :

- lapin ;
- cage ;
- date de début ;
- date de fin ;
- raison ;
- observation.

Exemple :

R034

C03

01/09 → 15/09

C07

15/09 → 30/09

C12

30/09 → 15/10

---

## 9. Raisons possibles d'un déplacement

Le système doit permettre notamment :

INITIAL\_ASSIGNMENT

BREEDING

MATERNITY

FATTENING

QUARANTINE

CLEANING

HEALTH

SALE

AGGRESSION

OVERCROWDING

SEX\_SEPARATION

SIZE\_DIFFERENCE

OTHER

Cela permet ensuite de savoir pourquoi les animaux ont été déplacés.

---

## 10. Gestion des lapins

Chaque lapin possède un identifiant technique UUID.

Il possède également un **code lisible par l'éleveur**.

Exemple :

R034

Pour les reproducteurs :

M01

M02

F10

F11

Le code doit être unique dans une ferme.

---

# 11. Identification physique des lapins

C'est une exigence majeure.

L'apparence physique ne constitue **pas** une identification fiable.

Deux lapins peuvent avoir :

- même race ;
- même couleur ;
- même sexe ;
- taille similaire ;
- caractéristiques physiques similaires.

L'application doit donc distinguer :

## Identifiant système

UUID

## Code élevage

R034

## Identification physique

TATTOO

RING

EAR\_TAG

OTHER

avec une valeur :

F10

ou :

Bague : 034

---

## 12. Identification temporaire des lapereaux

Les lapereaux peuvent ne pas recevoir immédiatement leur identification définitive.

À la naissance :

Portée L001

L001-01

L001-02

L001-03

...

L001-14

Ils sont associés à :

Mère : F10

Père : M02

Portée : L001

Plus tard, un lapereau sélectionné comme reproducteur peut recevoir son identité définitive :

L001-07

↓

R034

L'application doit conserver la correspondance.

---

## 13. Gestion des caractéristiques physiques

Les caractéristiques physiques sont enregistrées comme **informations complémentaires**.

Exemples :

- couleur ;
- taille ;
- particularité de l'oreille ;
- tache ;
- observation ;
- photographie éventuellement dans une version ultérieure.

Elles ne remplacent jamais l'identifiant unique.

---

## 14. Gestion des races

Une ferme peut utiliser plusieurs races.

Une race possède :

- nom ;
- description.

Exemple :

Californien

Néo-Zélandais

Géant des Flandres

...

---

## 15. Gestion des croisements

L'application doit permettre de conserver les croisements.

Exemple :

Race A : Néo-Zélandais

Race B : Californien

Croisement :

Néo-Zélandais × Californien

Le système doit conserver le type génétique :

PUREBRED

CROSSBRED

UNKNOWN

Le croisement exact doit être conservé.

---

## 16. Généalogie

Chaque lapin peut avoir :

fatherId

motherId

L'application doit pouvoir reconstruire :

Lapin

|— père

| |— père

| |— mère

|— mère

|— père

|— mère

Et également les descendants :

M02

|— R034

|— R041

|— R052

---

# 17. Principe essentiel de la généalogie

La généalogie est **indépendante des cages**.

Par exemple :

R034 → C01

R052 → C15

Cela ne change absolument pas le fait que :

R034

père = M02

mère = F10

R052

père = M02

mère = F10

Ils restent frères et sœurs.

---

# 18. Contrôle de la consanguinité

Avant tout accouplement, l'application doit effectuer une vérification généalogique.

Elle doit rechercher notamment :

## Relation parent/enfant

père × fille

mère × fils

→ blocage.

## Frères et sœurs

même père

+

même mère

→ blocage.

### **Demi-frères / demi-sœurs**

même père

OU

même mère

→ avertissement ou blocage selon la politique de l'élevage.

### **Ancêtre commun**

Le système doit pouvoir rechercher des ancêtres communs sur plusieurs générations.

Exemple :

R034

↓

M02

↓

M01

R052

↓

F11

↓

M01

→ ancêtre commun détecté.

---

## 19. Niveau avancé : coefficient de consanguinité

Une version ultérieure pourra calculer un **coefficient de consanguinité** à partir de la généalogie disponible.

Important :

Ce calcul ne remplacera pas une analyse génétique en laboratoire.

Il s'agit d'une estimation généalogique basée sur les ancêtres connus.

---

## 20. Analyses génétiques

L'application pourra ultérieurement enregistrer des résultats d'analyses génétiques réalisées par un laboratoire.

Exemple :

Lapin : R034

Type d'analyse :

Analyse génétique

Laboratoire :

...

Date :

...

Résultat :

...

Document :

...

L'application ne prétend pas réaliser elle-même le diagnostic génétique.

---

## 21. Gestion des accouplements

Un accouplement associe :

Femelle

+

Mâle

+

Date

Avant l'enregistrement :

1. vérifier que la femelle existe ;
  2. vérifier qu'elle est active ;
  3. vérifier que le mâle existe ;
  4. vérifier qu'il est actif ;
  5. vérifier les sexes ;
  6. vérifier l'appartenance à la même ferme ;
  7. vérifier la compatibilité généalogique ;
  8. enregistrer l'accouplement.
- 

## 22. Calcul automatique du calendrier de reproduction

Après un accouplement :

Jour 0

Accouplement

l'application calcule :

J+10 → début période de palpation

J+14 → fin période de palpation

J+28 → installation du nid

J+30 → début période probable de mise bas

J+32 → fin période probable de mise bas

Ces dates sont des **dates prévisionnelles**, pas des garanties biologiques.

---

## 23. Statuts d'un accouplement

PLANNED

AWAITING\_PALPATION

PREGNANT

NON\_PREGNANT

INCONCLUSIVE

BIRTH\_RECORDED

CANCELLED

---

## 24. Gestion de la gestation

Après palpation :

PREGNANT

NOT\_PREGNANT

INCONCLUSIVE

L'application enregistre :

- date ;
- résultat ;
- observation.

Si la femelle est vide :

accouplement terminé

Elle peut être remise dans le programme de reproduction selon les règles de l'éleveur.

---

## 25. Installation du nid

Vers J+28 :

Femelle F10

↓

Installation du nid

L'application crée une tâche :

Installer le nid de F10

L'éleveur peut enregistrer :

- date ;
  - type de nid ;
  - matériau ;
  - observation.
- 

## 26. Mise bas

À la naissance, l'éleveur enregistre :

- femelle ;
- accouplement ;
- date exacte ;
- nombre de lapereaux vivants ;
- nombre de mort-nés ;
- observations.

Exemple :

F10

10/09/2026

Vivants : 14

Mort-nés : 1

---

## 27. Gestion des portées

Une portée possède :

code

motherId

fatherId

birthDate

liveBorn

stillBorn

currentCount

status

Exemple :

L001

Mère : F10

Père : M02

Nés vivants : 14

Mort-nés : 1

---

## 28. Gestion des lapereaux d'une portée

Chaque lapereau peut être représenté individuellement lorsqu'un suivi individuel est nécessaire.

Exemple :

L001-01

L001-02

...

L001-14

Tous conservent :

mère = F10

père = M02

portée = L001

---

## 29. Répartition d'une portée dans plusieurs cages

Exemple :

Portée L001

14 lapereaux

Répartition :

C01 → 5 lapereaux

C02 → 5 lapereaux

C03 → 4 lapereaux

L'application doit pouvoir enregistrer cette répartition.

Mais elle doit conserver individuellement :

L001-01 → C01

L001-02 → C01

...

si les lapereaux sont suivis individuellement.

---

## 30. Mélange de plusieurs portées

L'application doit également permettre :

C08

L001-01

L001-02

L001-03

L002-01

L002-02

L002-03

Les animaux vivent dans la même cage mais conservent chacun leur généalogie.

Ainsi :

Cage ≠ portée

Cage ≠ parenté

Cage ≠ identité

---

## 31. Séparation des lapereaux agressifs

Lorsqu'un lapereau devient agressif :

L001-07

C02 → C05

le système crée un mouvement :

RabbitCageMovement

Rabbit : L001-07

From : C02

To : C05

Reason : AGGRESSION

Date : ...

La généalogie ne change pas.

---

## 32. Adoption / égalisation

Si une portée est trop importante :

L001 → 14 lapereaux

L002 → 7 lapereaux

l'éleveur peut déplacer :

2 lapereaux

L001 → L002

L'application doit enregistrer :

sourceLitter

destinationLitter

quantity

date

reason

Cela permet de savoir que les lapereaux élevés par une femelle ne sont pas nécessairement tous ses descendants biologiques.

C'est très important pour la généalogie.

---

## 33. Distinction biologique / mère adoptive

Il faut donc distinguer :

Mère biologique

de :

Mère nourricière / portée d'élevage

Exemple :

Lapereau R034

Mère biologique : F10

Père biologique : M02

Portée biologique : L001

Élevé temporairement dans :

L002

Cette distinction devra être conservée.

---

## 34. Sevrage

Vers J+35 :

Portée



Sevrage



Engraissement

L'application enregistre :

- date prévue ;
- date réelle ;
- quantité sevrée ;
- poids total ;
- poids moyen ;
- observation.

---

## 35. Lots d'engraisement

Les lapins destinés à la viande peuvent être gérés par lots.

Exemple :

Lot ENG-001

8 lapins

Entrée : 15/10

Les lapins peuvent appartenir à un lot tout en conservant leur identité individuelle.

---

## 36. Pesées

Deux modes :

**Pesée de lot**

8 lapins

60 jours

Poids total : 13,6 kg

Poids moyen : 1,70 kg

### **Pesée individuelle**

R034

60 jours

1,85 kg

Les deux modes doivent être supportés.

---

## **37. Mortalité**

Pour un lapin identifié :

R034

Date

Cause

Observation

Pour une portée :

Portée L001

Nés : 14

Morts : 2

Restants : 12

L'application doit conserver la cause lorsque celle-ci est connue.

---

## **38. Module santé**

Chaque animal peut avoir un historique :

Date

Symptômes

Observation

Diagnostic

Traitement

Médicament

Dosage

Prochaine échéance

---

## 39. Distinction observation / diagnostic

L'application doit distinguer :

### **Symptôme observé**

diarrhée

perte d'appétit

écoulement

etc.

### **Suspicion**

suspicion de ...

### **Diagnostic confirmé**

diagnostic vétérinaire

Cela évite de présenter une simple observation comme un diagnostic médical.

---

## 40. Analyses vétérinaires

Une future extension permettra d'enregistrer :

Type de prélèvement

Type d'analyse

Laboratoire

Date

Résultat

Document

Exemples d'analyses possibles selon indication vétérinaire :

- examen parasitologique ;
- analyse bactériologique ;
- PCR ;
- sérologie ;
- autres analyses de laboratoire.

L'application ne réalise pas elle-même ces analyses.

---

## 41. Ventes

Deux types principaux :

### Vente à l'unité

R034

Prix : ...

Client : ...

### Vente au poids

Lot ENG-005

Poids : 13,6 kg

Prix/kg : ...

Client : ...

---

## 42. Paiements

Une vente peut être :

UNPAID

PARTIALLY\_PAID

PAID

CANCELLED

Un client peut payer en plusieurs fois.

Méthodes :

CASH

MOBILE\_MONEY

BANK\_TRANSFER

OTHER

---

## 43. Clients

Un client peut avoir :

- nom ;
  - téléphone ;
  - email ;
  - adresse ;
  - historique des achats ;
  - crédit restant.
- 

## 44. Dépenses

Catégories :

FEED

HEALTH

MATERIAL

TRANSPORT

ENERGY

LABOR

OTHER

Chaque dépense contient :

- date ;
  - catégorie ;
  - description ;
  - montant ;
  - fournisseur ;
  - moyen de paiement ;
  - observation.
- 

## 45. Gestion des stocks

L'application ne doit pas seulement stocker :

Stock actuel = 500 kg

Elle doit conserver les mouvements.

Principe :

Stock initial

+

Achats

-

Consommation

-

Pertes

+

Ajustements

=

Stock actuel

---

## 46. Autonomie alimentaire

L'application doit pouvoir estimer :

Stock actuel

÷

consommation quotidienne moyenne

=

nombre de jours d'autonomie

Exemple :

Stock : 180 kg

Consommation moyenne : 10 kg/jour

Autonomie ≈ 18 jours

---

## 47. Système de tâches

Les événements métier génèrent des tâches.

Exemple :

Accouplement

↓

J+10

↓

Palpation

Accouplement



J+28



Installation du nid

Naissance



J+35



Sevrage

Traitement



Date suivante



Contrôle

---

## 48. Priorités des tâches

LOW

MEDIUM

HIGH

URGENT

Statuts :

PENDING

IN\_PROGRESS

COMPLETED

CANCELLED

---

## 49. Dashboard

Le dashboard doit être orienté **action**.

### 49.1 Aujourd'hui

Afficher :

- tâches urgentes ;
  - femelles à palper ;
  - nids à installer ;
  - mises bas prévues ;
  - mises bas récentes ;
  - sevrages ;
  - traitements ;
  - vaccinations.
- 

## 50. Population

Afficher :

Nombre total de lapins

Mâles reproducteurs

Femelles reproductrices

Lapereaux

Lapins en engraissement

Et éventuellement :

Cages disponibles

Cages occupées

## 51. Performances techniques

Afficher :

- lapereaux nés ce mois ;
  - taux de mortalité ;
  - taux de survie ;
  - taux de sevrage ;
  - taux de réussite des accouplements ;
  - nombre moyen de lapereaux par portée ;
  - poids moyen à différents âges ;
  - performances des reproducteurs.
- 

## 52. Performances génétiques

Afficher :

- femelles les plus productives ;
  - taux de survie par femelle ;
  - performances des descendants ;
  - reproducteurs potentiels ;
  - liens de parenté ;
  - croisements interdits ;
  - croisements présentant un risque.
- 

## 53. Finances

Afficher :

- dépenses du mois ;
- chiffre d'affaires ;
- ventes ;
- paiements reçus ;
- crédits clients ;
- résultat estimé ;
- dépenses alimentaires ;

- dépenses de santé.

Le coût de production du kilogramme de viande sera considéré comme **estimatif** tant que les données de consommation, de poids et de coûts sont suffisamment complètes.

---

## 54. Traçabilité complète d'un lapin

La fiche d'un lapin doit pouvoir raconter son histoire.

Exemple :

R034

|

|— Identité physique

|

|— Portée de naissance

|

|— Père

|

|— Mère

|

|— Frères / sœurs

|

|— Historique des cages

|

|— Pesées

|

|— Santé

|

|— Accouplements

|

|— Descendants

|

|— Lot d'engraissement

|

|— Vente / décès / réforme

C'est une fonctionnalité centrale de l'application.

---

## 55. Traçabilité d'une portée

Pour une portée :

L001

on doit pouvoir retrouver :

Mère biologique

Père biologique

Date de naissance

Nombre de vivants

Mort-nés

Lapereaux

Cages successives

Adoptions

Mortalités

Sevrage

Lot d'engraissement

Ventes

---

## 56. Règle fondamentale concernant les groupes

L'application doit accepter qu'un lapin puisse passer successivement par :

Portée

↓

Cage 1

↓

Cage 2

↓

Cage 3

↓

Lot d'engraissement

↓

Cage d'engraissement

↓

Vente

Tout en conservant :

Identité

Généalogie

Historique

---

## 57. Base de données principale

Les principales entités seront :

User

Farm

FarmMembership

Section

Cage

Breed

CrossBreed

Rabbit

RabbitCageMovement

Breeding

Pregnancy

Birth

Litter

LitterAdoption

Weaning

FatteningLot

LotMembership

LotCageMovement

Weighing

HealthRecord

Customer

Sale

SaleItem

Payment

Expense

InventoryItem

StockMovement

Task

---

## 58. Principes de modélisation

### UUID

Tous les identifiants techniques utilisent des UUID.

### **Pas de suppression physique systématique**

On privilégie :

ACTIVE

SOLD

DEAD

CULLED

LOST

plutôt que la suppression définitive des données importantes.

## Historique

Les changements importants sont conservés.

## Source de vérité

Les événements métier constituent la source de vérité.

Les informations calculées sont des projections.

---

# 59. Exemple de cycle complet

Prenons :

F10 × M02

## Jour 0

Accouplement

Le système calcule :

Palpation : J+10 → J+14

Nid : J+28

Mise bas prévue : J+30 → J+32

## J+12

Gestation confirmée

## J+28

Installation du nid

## J+31

14 lapereaux vivants

1 mort-né

## Ensuite

L001

↓

14 individus

↓

C01 / C02 / C03

### **Plus tard**

R001 → C01 → C04

R002 → C02

R003 → C03 → C06

...

### **J+35**

Sevrage

### **Engraissement**

ENG-001

### **À 60 jours**

8 lapins

13,6 kg

### **Sélection**

R003

↓

futur reproducteur

### **Accouplement futur**

L'application vérifie automatiquement sa généalogie avant d'autoriser le croisement.

---

## 60. Sécurité

L'application doit prévoir :

- authentification ;
- JWT ;
- mots de passe hashés ;
- contrôle d'accès par rôle ;
- isolation des données par ferme ;
- validation des entrées ;
- protection contre les accès non autorisés ;
- logs des opérations importantes.

Un utilisateur d'une ferme ne doit pas pouvoir consulter les données d'une autre ferme.

---

## 61. API GraphQL

L'API comportera notamment des domaines :

Auth

Users

Farms

Sections

Cages

Rabbits

Genetics

Breeding

Pregnancy

Births

Litters

Adoptions

Weaning

Fattening

Weighings

Health

Customers

Sales

Payments

Expenses

Inventory

Tasks

Dashboard

Reports

Chaque domaine pourra exposer :

Queries

Mutations

Inputs

Types

Resolvers

---

## 62. Exemple de mutation d'accouplement

```
mutation {  
  createBreeding(  
    input: {  
      femaleId: "..."  
      maleId: "..."  
      breedingDate: "2026-09-10"  
    }  
  )  
}
```

```
) {  
  id  
  status  
  breedingDate  
  palpationStartDate  
  palpationEndDate  
  nestDate  
  expectedBirthStartDate  
  expectedBirthEndDate  
}  
}
```

Le backend doit effectuer les contrôles métier avant de créer l'opération.

---

## 63. Exemple de contrôle généalogique

Avant :

F24 × M18

le backend analyse :

parents

↓

grands-parents

↓

arrière-grands-parents

↓

ancêtres communs

Puis retourne par exemple :

compatible

ou :

WARNING

Ancêtre commun détecté

ou :

BLOCKED

Relation parent/enfant détectée

---

## 64. PWA

L'application frontend sera conçue comme une PWA.

Elle devra :

- être responsive ;
- pouvoir être installée sur smartphone ;
- fonctionner correctement sur écran mobile ;
- utiliser un manifest ;
- disposer d'un service worker ;
- mettre en cache les ressources statiques.

### Important

La première version sera **online-first**.

Une véritable utilisation hors connexion nécessitera ultérieurement :

IndexedDB

+

synchronisation

+

gestion des conflits

Elle n'est donc pas incluse dans le MVP initial.

---

## 65. Gestion des erreurs

L'API doit retourner des erreurs compréhensibles.

Exemple :

Impossible d'enregistrer l'accouplement.

La femelle F24 et le mâle M18 ont un parent commun.

ou :

Impossible de déplacer 8 lapins.

La cage C04 a une capacité maximale de 6 lapins.

---

## 66. Validation métier

Les contraintes importantes doivent être contrôlées dans le backend.

Exemples :

- une femelle doit réellement être une femelle ;
  - un mâle doit réellement être un mâle ;
  - les deux animaux doivent appartenir à la ferme ;
  - un lapin vendu ne doit pas être réutilisé comme reproducteur actif ;
  - une cage ne doit pas dépasser sa capacité ;
  - un lapin ne peut pas avoir deux emplacements actifs simultanément ;
  - un lapin ne peut pas être son propre ancêtre ;
  - un accouplement interdit généalogiquement doit être bloqué ;
  - une adoption doit avoir une quantité valide ;
  - un paiement ne doit pas dépasser les règles financières définies ;
  - un stock ne doit pas devenir incohérent.
- 

## 67. Tests

Le projet devra comporter :

## Tests unitaires

Pour :

- calcul des dates ;
- généalogie ;
- consanguinité ;
- stock ;
- calculs de performances ;
- règles métier.

## Tests d'intégration

Pour :

- Prisma ;
- PostgreSQL ;
- GraphQL ;
- authentification.

## Tests fonctionnels

Pour les scénarios :

Accouplement → Gestation → Naissance → Portée → Sevrage

et :

Naissance → plusieurs cages → déplacement → engraissement → vente

---

# 68. MVP

Le MVP doit comprendre :

## Gestion

- authentification ;
- ferme ;
- utilisateurs ;
- sections ;
- cages.

## Animaux

- lapins ;

- identification ;
- portées ;
- généalogie ;
- mouvements de cages.

## **Reproduction**

- accouplements ;
- palpation ;
- gestation ;
- nid ;
- naissance ;
- adoption ;
- sevrage.

## **Production**

- lots ;
- pesées ;
- mortalité.

## **Santé**

- observations ;
- diagnostics ;
- traitements ;
- rappels.

## **Finance**

- clients ;
- ventes ;
- paiements ;
- dépenses.

## **Stock**

- aliments ;
- entrées ;
- consommations ;
- pertes.

## **Pilotage**

- dashboard ;
- tâches ;
- alertes ;
- statistiques principales.

## Génétique

- arbre généalogique ;
  - détection des liens proches ;
  - avertissement/blocage des croisements à risque.
- 

## 69. Fonctionnalités reportées

Pour éviter de rendre le premier projet trop complexe :

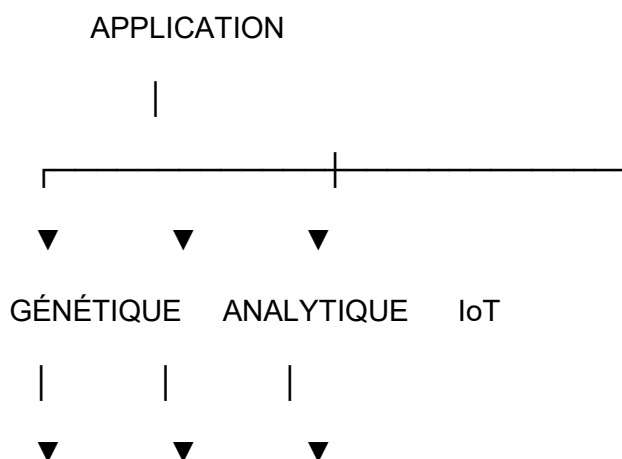
- application mobile native ;
- SMS ;
- WhatsApp ;
- paiement en ligne ;
- marketplace ;
- comptabilité complète ;
- IA prédictive ;
- recommandation automatique avancée des accouplements ;
- balance connectée ;
- analyse ADN directement connectée à l'application ;
- synchronisation offline avancée ;
- multi-fermes avancé.

Ces fonctionnalités pourront être ajoutées progressivement.

---

## 70. Évolution future

À terme, l'application pourra évoluer vers :



ADN/labo    prédictions    balances  
capteurs

On pourrait notamment ajouter :

- calcul avancé de consanguinité ;
- analyse des performances génétiques ;
- prévision de production ;
- recommandations de reproduction ;
- suivi automatique du poids ;
- intégration de balances ;
- analyse des maladies ;
- synchronisation hors ligne ;
- rapports avancés.

---

## 71. Principe directeur du projet

L'application doit respecter cette chaîne :

OBSERVATION

↓

ENREGISTREMENT

↓

HISTORIQUE

↓

ANALYSE

↓

ALERTE

↓

DÉCISION

Exemple :

Accouplement



date enregistrée



J+10 calculé



tâche de palpation



gestation confirmée



nid à J+28



mise bas



portée



sevrage



croissance



sélection



nouveau reproducteur



contrôle généalogique



nouvel accouplement

---

## 72. Règle d'or de la traçabilité

Le système doit toujours pouvoir répondre à ces questions :

**Qui est ce lapin ?**

**D'où vient-il ?**

**Où se trouve-t-il actuellement ?**

**Où était-il auparavant ?**

**Qui sont ses parents ?**

**Avec quels animaux est-il apparenté ?**

**Peut-il être utilisé pour la reproduction ?**

**Quelles portées a-t-il produites ?**

**Quelles sont ses performances ?**

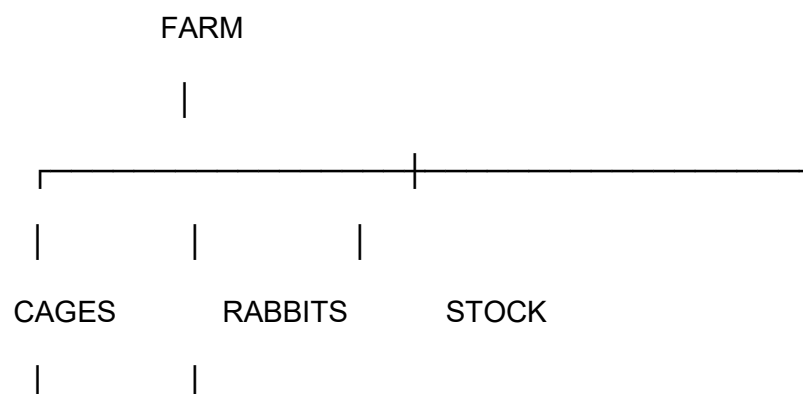
**Qu'est-il finalement devenu ?**

Si l'application peut répondre à ces questions, elle possède une vraie **traçabilité d'élevage**.

---

## Architecture finale du domaine

Le cœur du projet peut maintenant être résumé ainsi :





## La décision technique la plus importante après cette révision

Je ne passerais **pas encore directement à la migration Prisma**. Maintenant que nous avons clarifié les cas réels de ton élevage, la prochaine étape devrait être de transformer ce cahier des charges en **modèle de données définitif** : revoir chaque modèle Prisma, notamment *Rabbit*, *Litter*, *RabbitCageMovement*, *LotMembership*, la généalogie et les éventuelles tables d'identification/adoption, puis vérifier toutes les relations avant de lancer *prisma migrate*.

C'est préférable, parce qu'une modification de la structure maintenant est beaucoup moins coûteuse qu'une modification après avoir commencé à construire tous les resolvers GraphQL et services NestJS.